King Saud University
College of Computer & Information Sciences
Department of Computer Science

CSC339 – Theory of Computation

Course Project Report

NFA to DFA Conversion Tool

using the Subset Construction Algorithm

Group Members

#	Name	Student ID
1	Aleen Alfawzan	_____
2	_____	_____
3	_____	_____
4	_____	_____

Section: 2818 | Instructor: _____

Submission: November 2026

1. Project Overview

The goal of this project is an *Automata Conversion Tool* that takes a Nondeterministic Finite Automaton N = (Q, Σ, δ, q_0, F) and produces an equivalent Deterministic Finite Automaton D that recognises exactly the same language, using the **Subset Construction** algorithm. The tool is written in **Python 3**, uses **Graphviz** to draw state diagrams and **Streamlit** for a browser-based user interface. It supports ordinary NFAs as well as NFAs with **ε-transitions**.

Main features

- Enter an NFA in a form (Q, Σ, q_0, F and one transition rule per line such as `q0, a -> q0, q1`), load one of five ready examples, or upload a JSON file. Input is validated with clear error messages.
- One-click conversion showing the DFA states, initial and accepting states, the transition table, a step-by-step trace of the algorithm and a colour-coded state diagram (downloadable as PNG/SVG; the DFA can be saved as JSON).
- A string tester that runs each word on both the NFA and the DFA and shows the DFA path, as a live correctness check.
- A command-line version (`main.py`) and an automated test suite (`tests/`).

File	Responsibility
automata.py	NFA/DFA classes, parsing, validation, ε-closure, move, subset construction
visualize.py	Builds the Graphviz state diagrams of the NFA and the DFA
app.py	Streamlit graphical user interface
main.py	Command-line interface (prints the table and saves diagrams)
examples/*.json	Five test NFAs
tests/test_conversion.py	Automated validation (11 unit tests)

2. Subset Construction Implementation

Input representation

An NFA is stored in the `NFA` class: a list of states, a list of alphabet symbols, the start state, a set of accepting states and a dictionary `transitions[(state, symbol)] → set of states`. A missing key means δ = ∅, and the special symbol ε (typed as `eps`) marks empty moves. Before converting, `validate()` checks that the start and accepting states belong to Q, that every transition uses symbols in Σ ∪ {ε} and targets existing states. Each DFA state is a `frozenset` of NFA states, so it can be used as a dictionary key and compared for equality.

Algorithm

Two helper operations are used: **move(R, a)** = $\cup_{r \in R}$ δ(r, a), and the **ε-closure E(R)**, computed with a stack (depth-first search over ε-edges). The DFA is then built with a worklist (queue), so only subsets that are **reachable** from the start state are generated instead of all $2^{|Q|}$ subsets:

```
start  = E({q0})
states = [start];  queue = [start]
while queue is not empty:
    R = queue.pop_front()
    for a in Σ:
        U = E( move(R, a) )
        if U not in states:  states.append(U);  queue.push(U)   # new DFA state
        δ'(R, a) = U
F' = { R in states :  R ∩ F ≠ ∅ }
```

Initial state: E({q_0}) (equal to {q_0} when there are no ε-moves). **Accepting states:** every subset that contains at least one accepting NFA state. **Transition function:** δ′(R, a) = E(move(R, a)), filled for every state and every symbol, so δ′ is total. When a subset has no move on a symbol the result is ∅, which becomes a **dead (trap) state** looping to itself; it is drawn dashed in the diagrams. Every step is recorded so the interface can show the full trace. The worst

case is $O(2^{|Q|}\cdot|\Sigma|\cdot|Q|^2)$, but in practice only a few reachable subsets appear.

Correctness check

The NFA class also has an independent simulator `accepts(w)` that tracks the set of current NFA states directly. The test suite compares the NFA and the generated DFA on **every string of length 0–10** over Σ, compares the DFA with a regular expression describing the intended language, checks that the DFA is complete, and compares Test 2 with the DFA given in the textbook (Sipser, Example 1.41). All 11 tests pass.

3. Screenshots of the Tool

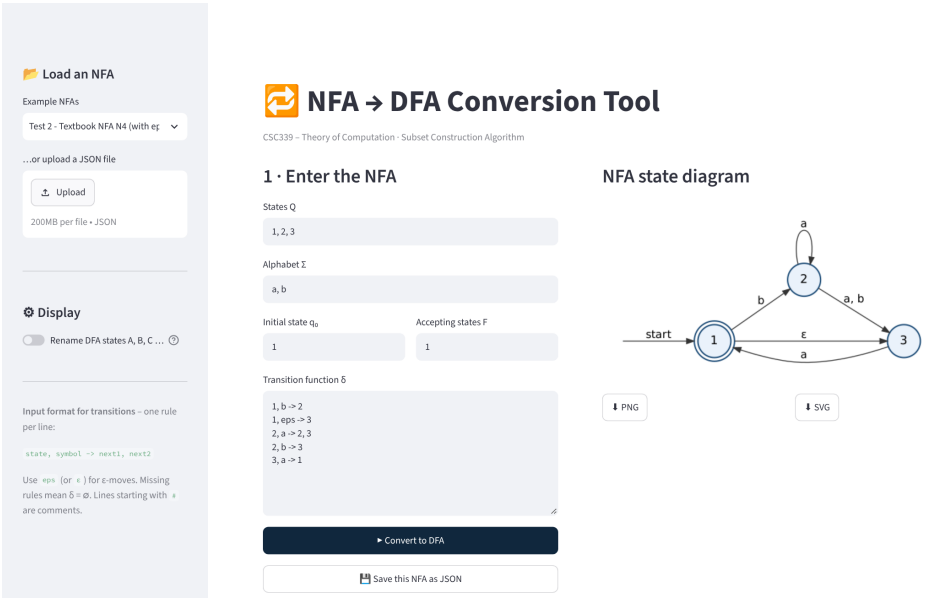


Figure 1 – Input form (left) with the live NFA diagram (right); examples and JSON upload are in the sidebar.

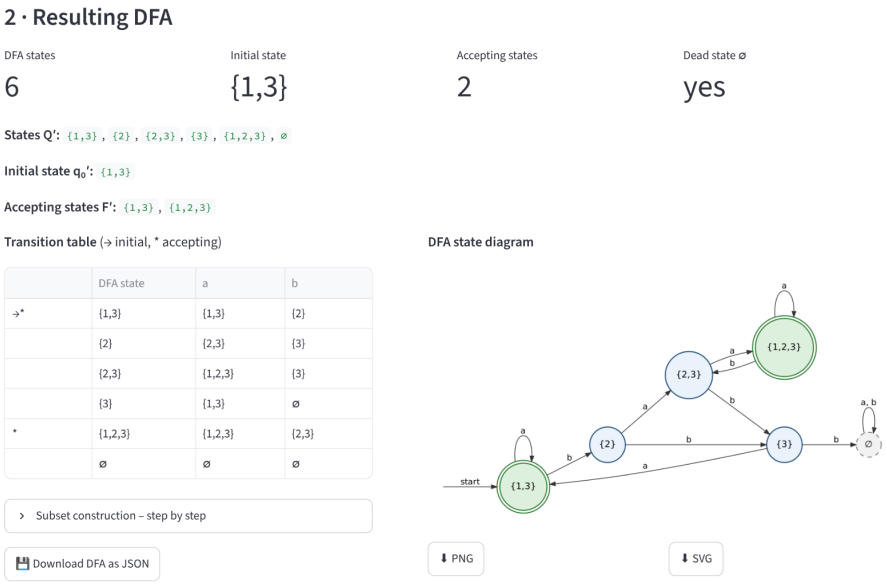


Figure 2 – Result after pressing *Convert to DFA*: DFA states, initial/accepting states, transition table and state diagram.



Figure 3 – String tester: each word is run on the NFA and on the DFA, with the DFA path shown.

4. Test Cases and Results

Five NFAs are included in `examples/`; three are presented here. (Tests 3 – "second-to-last symbol is 1" – and 5 – "contains aba" – are also in the project and pass all checks.)

Test Case 1: strings over {0,1} that end with 01

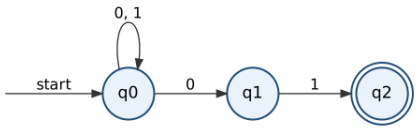
A plain NFA (no ε-moves) where q0 non-deterministically guesses where the final "01" begins. The DFA needs only three subsets; {q0,q2} is accepting because it contains q2.

NFA transition table

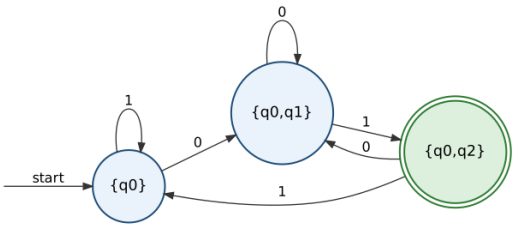
	state	0	1
→	q0	{q0,q1}	{q0}
	q1	∅	{q2}
*	q2	∅	∅

DFA transition table (→ initial, * accepting)

	DFA state	0	1
→	{q0}	{q0,q1}	{q0}
	{q0,q1}	{q0,q1}	{q0,q2}
*	{q0,q2}	{q0,q1}	{q0}



Input NFA



Generated DFA

Result: 3 reachable DFA states; initial state {q0}; accepting states {q0,q2}. **Sample strings:** 01 → accept; 1101 → accept; 10 → reject; ε → reject; 0 → reject. NFA and DFA agree on all strings up to length 10 ✔.

Test Case 2: textbook NFA N4 (with an ε-move)

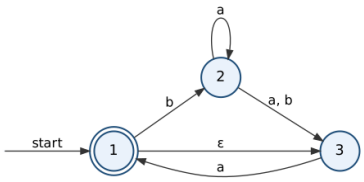
N4 from Sipser, Example 1.41. The ε-move 1→3 makes the start state E({1}) = {1,3}. Some states have no transitions on a symbol, so the empty set ∅ appears as a dead state. The 6 reachable states and the whole table match the textbook solution exactly (the remaining 2 of the 8 subsets are unreachable and are not generated).

NFA transition table

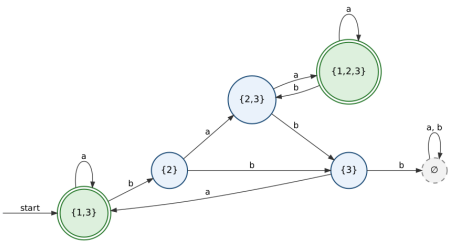
	state	a	b	ε
→*	1	∅	{2}	{3}
	2	{2,3}	{3}	∅
	3	{1}	∅	∅

DFA transition table (→ initial, * accepting)

	DFA state	a	b
→*	{1,3}	{1,3}	{2}
	{2}	{2,3}	{3}
	{2,3}	{1,2,3}	{3}
	{3}	{1,3}	∅
*	{1,2,3}	{1,2,3}	{2,3}
	∅	∅	∅



Input NFA



Generated DFA

Result: 6 reachable DFA states; initial state {1,3}; accepting states {1,3}, {1,2,3}. **Sample strings:** ε → accept; a → accept; baba → accept; baa → accept; bb → reject; abab → reject. NFA and DFA agree on all strings up to length 10 ✔.

Test Case 3: 0^k where k is a multiple of 2 or of 3 (ε-NFA)

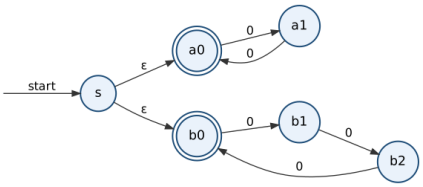
The start state s branches by ε-moves into a 2-cycle (a0, a1) and a 3-cycle (b0, b1, b2), i.e. the union of two languages. The DFA tracks both cycles at once (a pair (k mod 2, k mod 3)), giving a cycle of length 6 plus the start state.

NFA transition table

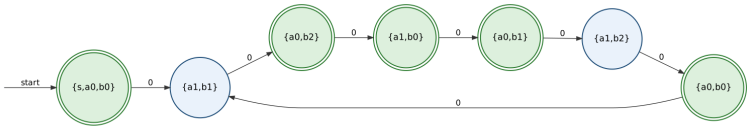
	state	0	ε
→	s	∅	{a0,b0}
*	a0	{a1}	∅
	a1	{a0}	∅
*	b0	{b1}	∅
	b1	{b2}	∅
	b2	{b0}	∅

DFA transition table (→ initial, * accepting)

	DFA state	0
→*	{s,a0,b0}	{a1,b1}
	{a1,b1}	{a0,b2}
*	{a0,b2}	{a1,b0}
*	{a1,b0}	{a0,b1}
*	{a0,b1}	{a1,b2}
	{a1,b2}	{a0,b0}
*	{a0,b0}	{a1,b1}



Input NFA



Generated DFA

Result: 7 reachable DFA states; initial state {s,a0,b0}; accepting states {s,a0,b0}, {a0,b2}, {a1,b0}, {a0,b1}, {a0,b0}. **Sample strings:** ε → accept; 0 → reject; 00 → accept; 000 → accept; 00000 → reject; 000000 → accept. NFA and DFA agree on all strings up to length 10 ✔.

Automated validation

Test	NFA |Q|	DFA |Q′|	Strings	NFA = DFA	Matches expected
1 – ends with 01	3	3	2 047	✔	✔ regex [01]*01
2 – textbook N4 (ε)	3	6	2 047	✔	✔ textbook table
3 – 2nd-last is 1	3	4	2 047	✔	✔ regex [01]*1[01]
4 – unary mod 2 or 3 (ε)	6	7	11	✔	✔ k mod 2 = 0 or k mod 3 = 0
5 – contains aba	4	6	2 047	✔	✔ regex [ab]*aba[ab]*

Strings checked = all words of length 0–10 over Σ. Further unit tests cover invalid input (unknown states/symbols, bad syntax), all spellings of ε, and that unreachable subsets are not generated.

5. Conclusion

The tool correctly implements the subset construction for NFAs with and without ε-moves, generating only reachable DFA states, a complete transition function with a dead state when needed, and clear tables and diagrams. Exhaustive comparison between the NFA and the generated DFA on all short strings, together with the textbook example, confirms that every generated DFA recognises the same language as its NFA.